Nonparametric estimation of a distribution function and of the density derived from it

Parzen window and neural network: method, measurements, and limits

`parzen-cdf` project

September 8, 2026

Abstract

Given a finite sample and no assumption about the distribution behind it, we estimate the cumulative distribution function with a small neural network trained on Parzen-window labels, and obtain the density by differentiating it. The report is self-contained: every quantitative claim states how it was obtained, and the scripts that produce each number are listed in Appendix B.

Three features of the procedure are not the obvious choices, and the report argues for each rather than asserting it. The window width is chosen by cross-validation instead of a rule of thumb, because a rule of the form $h_1 = c\,\hat{\sigma}$ cannot be made to work: on a standard benchmark the optimal constant ranges over a factor of twelve between distributions, and calibrating it more carefully makes the rule worse. The network is written so that its output is a valid distribution function at every value of its parameters, which puts monotonicity, the limits at infinity, the positivity of the density and its unit mass into the model rather than into a repair step applied to a grid afterwards. And because the distribution is unknown on the data the method is built for, quality is reported through quantities computable from the sample alone; we measured which of those are informative, and the one a reader would most expect to see turns out to be anticorrelated with the error it appears to certify.

On five multimodal targets at $n = 500$ the procedure improves on a conventional neural design and on the Parzen estimator it learns from. One limitation is stated rather than left implicit: a mixture of logistic distribution functions has strictly positive density on the whole line, so distributions with sharp edges are rounded off and the error grows by a factor between three and fourteen. Adding capacity halves that error and costs a third of the accuracy on the multimodal targets, so we do not take it.

Contents

1 Introduction

1.1 The problem

We are given n observations $x_1, \ldots, x_n$, drawn independently from a distribution F that we know nothing about, and we are asked for two things: an estimate $\widehat{F}$ of the distribution function, and an estimate $\widehat{f}$ of the density, obtained by differentiating the first.

The order matters, and it is not incidental. One could estimate the density directly and integrate it. Going the other way round, estimating F first and differentiating, is what the assignment prescribes, and there is a technical reason for it as well. A distribution function is a smoother object than a density: it is an integral, and integration averages away exactly the kind of local fluctuation that makes density estimation hard. A finite sample produces a jagged density estimate but a rather well behaved cumulative one. The price is paid at the end, when differentiating brings the fluctuation back, and much of this report is about paying that price as little as possible.

There is a second reason, specific to the neural approach. Regressing a curve that is bounded between 0 and 1, monotone, and asymptotically flat is a far better conditioned problem than regressing a density, which is unbounded above, has no natural scale, and must integrate to one. If the network is asked for a distribution function, its structure can be made to guarantee all the properties we want; if it is asked for a density, guaranteeing non-negativity and unit mass at the same time is considerably harder.

1.2 The configuration

What the pipeline returns is $\widehat{F}$ and $\widehat{f}$ as functions, evaluable at any real point rather than tabulated on a grid. The settings that produce them are collected in Table 1, each with the section that argues for it. They are the defaults of the delivered entry point, so a run that specifies nothing uses exactly this configuration.

Stage	Setting	Where
Window	Least-squares cross-validation, reported as $\hat{h}_1 = \hat{h}\sqrt{n}$	§4
Labels	Leave-one-out Parzen distribution function at the sample points, computed with half the selected window; wider labels erase the structure the network is there to recover	§6.2
Estimator	Convex mixture of logistic distribution functions on a standardised input; twelve of them, chosen on the worst case of the capacity table	§5.8
Initialisation	Component centres at the sample quantiles, equal weights; consumes no randomness	§6.4
Optimiser	Adam, step 0.03, 6000 full-batch steps, squared error against the labels	§6
Density	Derivative of the mixture in closed form	§7
Domain	$[\min x - 3\hat{\sigma},\ \max x + 3\hat{\sigma}]$, for drawing and integrating only	§8
Diagnostics	LSCV score, leave-one-out log-likelihood, integrated mass	§8
Deliberately absent	Rectification, monotonicity penalty, density clamp, and any comparison against the empirical distribution function	§6.3, §8

Table 1: The delivered configuration. Appendix C gives the evidence behind each choice.

Proofs are collected in Appendix A, and Appendix B lists the script that reproduces each measurement reported here.

2 Development data: generating honest samples

2.1 Why the generator matters even though it is not part of the pipeline

The estimator never sees the generator. It receives a vector of numbers and knows nothing about where they came from, which is the whole point of a nonparametric method.

The measurements are a different matter. Every number in this report was taken on samples we generated ourselves, because calibrating the window, choosing the number of network components and comparing architectures all need a distribution whose truth is known. A generator with a subtle bias would contaminate all of it without anything downstream looking wrong, so it gets the same scrutiny here as the estimator.

2.2 Ancestral sampling from a mixture

Our test distributions are mixtures: densities of the form

$$f(x) \;=\; \sum_{i=1}^{k} w_i \, f_i(x), \qquad w_i \ge 0, \quad \sum_i w_i = 1, \tag{1}$$

where the components f_i are ordinary densities, typically normal. Mixtures are convenient because their distribution function is the same weighted sum of the component distribution functions, so the reference curves against which we score are available in closed form.

A mixture is best understood not as a formula but as a two-stage random process, with a hidden variable: first draw which component will generate the point, then draw the point from

that component. Sampling by following those two stages in order is called *ancestral sampling*, a name borrowed from graphical models, where one draws each variable after its parents. In our case the parent is the component index I and the child is the observation X:

$$I \sim \text{Categorical}(w_1, \ldots, w_k), \qquad X \mid I = i \;\sim\; f_i.$$

The construction gives back the mixture exactly:

$$\Pr(X \le x) \;=\; \sum_i \Pr(I = i)\, \Pr(X \le x \mid I = i) \;=\; \sum_i w_i F_i(x),$$

which is the mixture distribution function by definition. There is no tolerance, no convergence, no numerical error. The cost is $O(n)$: one categorical draw and one component draw per observation, both vectorised.

The alternative would be *inverse transform sampling*: if U is uniform on $(0, 1)$ then $F^{-1}(U)$ has distribution function F, for any F whatsoever. That generality is what makes it the standard fallback, and here it is what it costs. The distribution function of a mixture of normals cannot be inverted in closed form, so each sample would require a numerical root search. The result would be exact only up to the tolerance of that search, and would cost $O(n \times \text{iterations})$. We implemented it anyway and compared: the two samples are statistically indistinguishable (two-sample Kolmogorov–Smirnov test, $p = 0.696$). Both are correct, then; what separates them is that one is exact and cheap and the other approximate and expensive.

2.3 The Ziggurat algorithm

Ancestral sampling reduces the problem to drawing from a single normal component, which leaves the question of how a normal variate is produced from uniform random bits. This is a classical problem with several classical answers, and the efficiency of the whole simulation depends on which one is used.

The obvious approach, inverting the normal distribution function, requires evaluating a special function for every sample. The Box–Muller transform produces two normal variates from two uniforms using a logarithm, a square root and a pair of trigonometric functions; Marsaglia's polar method removes the trigonometry at the cost of a rejection step. All of them are exact; all of them are relatively expensive per sample.

The *Ziggurat* method (Marsaglia and Tsang, 2000) is a rejection sampler designed so that the expensive branch is almost never taken. Consider the right half of the bell curve and cover it with a stack of horizontal strips, around 128 or 256 of them, chosen to all have *exactly the same area*, plus a tail region at the bottom with that same area. The picture resembles a stepped Mesopotamian ziggurat, which is where the name comes from. Because the strips have equal area, choosing one uniformly at random is a matter of masking a few random bits.

Within a strip, the density curve enters from one side: part of the rectangle lies entirely underneath the curve, and a thin fringe near the edge sticks out above it. The algorithm draws a strip index, draws a horizontal coordinate uniformly inside the strip, and then checks a single inequality: if the point falls in the safe region it is accepted immediately, with no evaluation of the exponential at all. Only if it falls in the fringe does the algorithm evaluate the density and accept with the appropriate probability; and only if the bottom strip is drawn does it fall back to a separate routine for the tail. With 256 strips the fast branch covers roughly 98–99% of draws, which is why the method costs little more than one uniform and one comparison per sample.

The point that matters for us is that this is a rejection sampler, and rejection sampling is *exact*: the accepted points are distributed precisely according to the target density, because the procedure is equivalent to sampling uniformly under the curve and keeping the abscissa. The speed is not bought with accuracy, and there is no approximation to declare.

The two levels are easy to confuse, so they are best separated. The Ziggurat is not something we implemented: it is the algorithm NumPy uses internally when it draws a standard normal variate. We verified this rather than assuming it. The documentation of `standard_normal` does not name the algorithm, but the compiled library that `Generator` links against exports symbols named after it, among them the tail cut-off constant of the normal Ziggurat and its reciprocal, while the legacy `RandomState` module exports no such symbol, consistent with its use of the polar method.

The two are also not alternatives to one another: they sit at different levels and compose. Ancestral sampling decides *which* normal to draw from; the Ziggurat is how each individual normal variate is actually produced.

2.4 Checking the generator

What can and cannot be verified should be stated separately. That NumPy implements the Ziggurat correctly is not something we can prove; what we can check, and what actually matters, is that the output has the properties we rely on: that it follows the intended distribution, that the components appear with the intended probabilities, and that successive draws are independent.

Check	**Method**	**Result**
Fit to the true CDF	KS test, 300 replicates	rejections at 5%: 0.050 (n=500), 0.047 (n=2000)
Uniformity of p-values	KS test on the p-values	passed (see below)
Component frequencies	χ^2 against the weights	$p = 0.533$; observed 0.3003/0.5011/0.1986
Serial independence	autocorrelation, lags 1–100	all within ± 0.0044; runs test $z = -0.93$
Sampling strategy	ancestral vs. inversion	indistinguishable, $p = 0.696$

Table 2: Checks on the sample generator. The second row is the one that matters most.

The second check is the one usually omitted. A correct generator should not merely *pass* a goodness-of-fit test; the p-values it produces should be *uniformly distributed*. A generator whose p-values cluster near one would be as suspicious as one that rejects too often, since it would mean the samples fit the target better than chance allows. Testing the uniformity of the p-values is therefore the right check, and it is the one we ran.

One further check concerns us directly. Every experiment in this report averages over repetitions obtained with seeds $0, 1, 2, \ldots$, and if nearby seeds produced correlated streams then those repetitions would not be independent and every reported average would be optimistic. With 300 replicates a few borderline values appeared, so we repeated the test with 2000:

Seeds	**Rej. 5%**	**Median p**	**Uniformity**	**Corr.**
$0 \ldots 1999$ (what we use)	0.0475	0.4912	0.834	-0.0009
random 64-bit	0.0405	0.4946	0.651	-0.0066
$10^6 \ldots 10^6{+}1999$	0.0505	0.5117	0.624	-0.0185

Table 3: Consecutive small seeds introduce no structure. The 95% band for the correlation is ± 0.0438; all three schemes sit well inside it.

The earlier borderline values were fluctuations. As an independent confirmation, the average of the 2000 sample means is 0.69930 against a true mixture mean of 0.70000, a discrepancy of about a third of a standard error.

3 The Parzen window estimator

3.1 Definition and intuition

The Parzen window estimator replaces each observation by a small bump and averages them. Given a kernel K, a density usually symmetric and centred at zero, and a window width $h > 0$, the density estimate is

$$\widehat{f}(x) \;=\; \frac{1}{nh}\sum_{i=1}^{n} k\left(\frac{x - x_i}{h}\right), \tag{2}$$

where k is the kernel density. The intuition is that a sample tells us where probability mass is, but says nothing about how it is spread in between; the kernel supplies that missing smoothness, and h decides how much of it.

The window is the only real choice. Too small, and the estimate degenerates towards a collection of spikes at the observations, reproducing the sample rather than the distribution; too large, and genuine structure is smoothed away, with separate modes merging into one. Everything else, the shape of the kernel and the details of the implementation, matters far less. Chapter 4 is devoted to this single number.

3.2 The logistic kernel and a closed-form distribution function

Since what we want first is the distribution function rather than the density, it pays to choose a kernel whose antiderivative we know. Integrating (2) gives

$$\widehat{F}(x) \;=\; \frac{1}{n}\sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right), \tag{3}$$

with K the kernel's own distribution function. For the *logistic* kernel that function is the sigmoid, $K(u) = 1/(1 + e^{-u})$, and the estimate becomes a plain average of sigmoids: smooth, strictly increasing, bounded between 0 and 1, and available in closed form at any point. Its derivative is exactly (2), so the density and the distribution function are consistent by construction rather than by numerical differentiation. We verified this to machine precision: the largest discrepancy between the analytic kernel density and the numerical derivative of the sigmoid is 1.3×10^{-8} over a fine grid.

That the estimate is a sum of sigmoids will matter again in Chapter 5, where the chosen network architecture turns out to have exactly the same form.

The choice of kernel carries a subtlety that is easy to miss, and we did miss it at first. Kernels differ in how much they smooth at a given h, because they have different spreads:

Kernel	**Standard deviation**
logistic	$\pi/\sqrt{3} \approx 1.8138$
Gaussian	1
box (uniform)	$1/\sqrt{3} \approx 0.5774$
Epanechnikov	$1/\sqrt{5} \approx 0.4472$
triangular	$1/\sqrt{6} \approx 0.4082$

Table 4: The available kernels and their spreads. Classical bandwidth rules assume the Gaussian row.

Every classical bandwidth rule (Silverman's, Scott's, the plug-in family) is derived for a kernel of unit standard deviation. Using such a rule with the logistic kernel and no correction therefore smooths 1.814 times more than intended. The fix is to divide the recommended width by the kernel's standard deviation, so that the two kernels smooth equivalently; we refer to this

below as *variance matching*. The cost of omitting it is not academic: on the reference trimodal mixture at $n = 500$, averaged over ten samples, the uncorrected rule gives a KS distance of 0.0628 against 0.0449, and an integrated squared error of 0.0334 against 0.0154.

3.3 The window must shrink as the sample grows

For the estimator to converge to the true density, the window must satisfy two classical conditions: $h_n \to 0$, so that the smoothing eventually vanishes, and $nh_n \to \infty$, so that the number of observations effectively contributing to each point still grows. The first alone would leave a spike at every sample point; the second alone would leave a permanently oversmoothed estimate.

The schedule used throughout this project,

$$h_n \;=\; \frac{h_1}{\sqrt{n}}, \tag{4}$$

satisfies both: $h_n \to 0$ and $nh_n = h_1\sqrt{n} \to \infty$. It reduces the choice of a sequence of windows to the choice of a single number h_1, which is why it is convenient, and Chapter 4 is about how that number should be chosen. We keep this form throughout, and Section 4.6 quantifies what it costs relative to the rate that would be optimal.

3.4 How an estimate is scored

Because the test distributions are known, every estimate can be compared against the truth. We use two measures, and they answer different questions.

The **Kolmogorov–Smirnov distance** $\sup_x |\widehat{F}(x) - F(x)|$ scores the distribution function. It has a natural reference point: the empirical distribution function, which uses no smoothing at all, has expected KS distance $\sqrt{\pi/2}\,\ln 2/\sqrt{n} \approx 0.8687/\sqrt{n}$. No estimator can reliably do much better than this, so it serves as a statistical floor: at $n = 500$ it is 0.0388.

The **integrated squared error** $\int(\widehat{f} - f)^2$ scores the density. This is the measure that matters most here, since the density is what the assignment ultimately asks for, and it is far less forgiving than the first. Integration is a smoothing operation, so errors that are plainly visible in a density estimate can be almost invisible in its integral. We will encounter a case where a badly chosen window costs a factor of two on the KS distance and a factor of seven and a half on the integrated squared error; judging that window by the first number alone would have declared it acceptable.

Two derived quantities are used when comparing selectors. The **oracle** is the window that minimises the true error for the sample at hand. It is not a selector, since it uses the answer, and it is never part of any procedure we propose; it serves only as a normaliser, since absolute errors are not comparable across distributions with different shapes. The **relative efficiency** of a selector is its error divided by the oracle's, a number at least 1, where 1 means it found the best window achievable with that estimator on that sample. Reporting efficiencies rather than raw errors is what makes it meaningful to average across a benchmark of very different densities.

4 The window: the central problem

4.1 The rule of thumb, and why it could not work

The natural way to fix h_1 is to tie it to a scale estimated from the data, and the natural scale is the sample standard deviation. Sweeping h_1 over three orders of magnitude on a ladder of Gaussian mixtures and picking a value that works across them gives $h_1 = 1.5\,\hat{\sigma}$, which is what we used for some time. The result looked convincing, and on the distributions it was tested against it was.

Three separate reasons make a rule obtained that way untrustworthy, and only the last of them is about the number itself.

The first is that the benchmark was also the prize. The constant was tuned on a handful of Gaussian mixtures and then validated on the same family, including ten mixtures drawn at random with component means spread uniformly over $[-5, 5]$ and standard deviations over $[0.2, 1.5]$. That procedure measures how well one has interpolated one's own test set, not how well the rule transfers to a distribution nobody chose.

The second is that $\hat{\sigma}$ is the wrong quantity to anchor to. The standard deviation measures the *global* spread of the sample, whereas the window has to resolve the *narrowest feature* of the density. On a smooth unimodal target these coincide, which is why the rule looks reasonable at first. On a multimodal target with well separated modes they diverge sharply: $\hat{\sigma}$ is inflated by the distance between the modes, while the window that resolves each mode has to shrink. Since the assignment explicitly concerns a strongly multimodal distribution, this is not a corner case.

The third is that the constant was validated on the Kolmogorov–Smirnov distance, which is the forgiving measure. On a mixture of five narrow well separated modes at $n = 1000$, the rule gives a KS distance twice the oracle's, which looks tolerable, and an integrated squared error seven and a half times the oracle's, which does not. Judging a window by the distance between distribution functions systematically underestimates how bad it is for the density, and the density is what we are asked to deliver.

4.2 An external test bench

To avoid repeating the first mistake, the study below uses a benchmark that we did not design. Marron and Wand (1992) introduced a set of fifteen normal mixtures specifically to stress bandwidth selectors, and it has been the standard reference in that literature ever since. The set is deliberately unkind: alongside a plain Gaussian and mild bimodals it contains the *claw*, a broad normal with five narrow spikes sitting on top of it, and its double and asymmetric variants, in which a narrow structure hides inside a much wider one. These are exactly the shapes on which a globally anchored rule fails.

We use the thirteen of them whose parameters we could verify against a primary source, the implementation shipped with the R package `nor1mix`; the two comb densities were left out because we could not confirm their parameters, and a mistyped density would invalidate everything measured on it. To these we add three Gaussian mixtures of our own, of the kind the assignment describes, so that the benchmark also covers the case of direct interest. Sixteen densities in total, all normal mixtures, so pdf and CDF are available in closed form and every estimate can be scored against the truth to machine precision.

4.3 The selectors compared

Every selector below sees the sample and nothing else. The classical rules are derived for a unit-variance kernel, so each is divided by $\pi/\sqrt{3}$ before being used with the logistic one, as explained in Section 3.4; without that correction they would all oversmooth by the same factor and the comparison would be meaningless.

Selector	Form
constant start	$h = 1/\sqrt{n}$, an untuned reference point
σ-rule	$h = 1.5\,\hat{\sigma}/\sqrt{n}$, the rule to beat
Silverman	$0.9\,\min(\hat{\sigma},\ \mathrm{IQR}/1.349)\,n^{-1/5}$
Scott	$1.06\,\hat{\sigma}\,n^{-1/5}$
two-stage plug-in	estimates $\int (f'')^2$ from the data, pilot from the normal reference
LSCV	minimises an unbiased estimate of the integrated squared error
MLCV	maximises the leave-one-out log-likelihood
$c\,\hat{\sigma}/\sqrt{n}$	the form of the σ-rule, with c recalibrated
$c\,s\,n^{-1/5}$	correct exponent, robust scale, c calibrated
$c \cdot d_k$	anchored to a *local* scale, the distance to the k-th neighbour

Table 5: The selectors compared. The last three have a free constant.

We derived the plug-in rather than copying a formula. The asymptotically optimal window depends on $\theta_4 = \int (f'')^2$, a functional of the unknown density; estimating θ_4 with a kernel estimator requires a pilot bandwidth, which we take from the normal reference. The derivation is in Appendix A; as a check, feeding the normal reference into both stages reproduces Scott's rule exactly, and on Gaussian samples the implemented selector returns an implicit constant of 1.043 to 1.051 against the theoretical 1.06.

The three selectors with a free constant raise an obvious objection: calibrating them on the benchmark and then scoring them on the same benchmark would repeat precisely the error we are criticising. They are therefore calibrated *leave-one-density-out*: the constant used on any given density is the one that performs best on all the others. It is the least that can be asked of a rule before claiming it generalises.

Errors are reported as efficiencies relative to the oracle, as defined in Section 3.4, since absolute errors are not comparable across densities with such different shapes.

One caveat belongs here rather than in the discussion of the results. LSCV minimises an unbiased estimate of exactly the criterion we score it on, so an advantage on the integrated squared error is expected and proves little by itself. The independent check is the KS column, where its objective has no structural advantage.

4.4 The central result: the constant is not a constant

Comparing selectors comes second. First we checked whether the rule we had been looking for exists at all, by computing, for each family with a free constant, the optimal value separately on each density. If a law of the form $h_1 = c\,\hat{\sigma}$ existed, those values would agree.

Family	Smallest	Largest	Ratio
$c\,\hat{\sigma}/\sqrt{n}$, the form of the σ-rule	0.36	4.15	11.6×
$c\,s\,n^{-1/5}$, correct exponent and robust scale	0.11	1.16	10.6×
$c \cdot d_k$, anchored to a local scale	0.46	2.54	5.6×

Table 6: Optimal constant, density by density, at $n = 500$. At $n = 1000$ the first row spreads further, from 0.40 to 5.31, a ratio of 13.2.

No single number can sit at 0.36 and at 4.15 simultaneously. The value 1.5 is not a badly chosen constant; the form of the rule is what fails. It is also visible that anchoring to a local scale rather than to $\hat{\sigma}$ halves the spread, which confirms the diagnosis of Section 4.1 without rescuing the idea.

The decisive check is what happens when the constant is recalibrated properly. Using leave-one-density-out calibration on the full sixteen-density benchmark, the recalibrated σ-rule reaches a mean efficiency of 2.46, against 2.27 for the original 1.5 left untouched. Fitting the constant on a larger and more varied benchmark makes the rule *worse*, because there is no constant to find: the calibration merely trades one set of densities for another.

4.5 Least-squares cross-validation, and its measured instability

Selector	Mean ISE	90th pct	Worst	Mean KS	Cases $> 2\times$
constant start	2.66	5.07	6.39	1.28	51%
σ-rule ($1.5\,\hat{\sigma}$)	2.27	3.95	9.38	1.25	33%
Silverman	3.77	8.11	21.01	1.49	46%
Scott	5.27	12.33	22.94	1.84	54%
two-stage plug-in	3.13	6.95	18.77	1.37	36%
LSCV	**1.26**	**1.61**	**1.59**	**1.11**	**6%**
MLCV	1.37	1.98	3.11	1.18	10%
$c\,\hat{\sigma}/\sqrt{n}$ recalibrated	2.46	5.33	9.89	1.23	34%
$c\,s\,n^{-1/5}$ recalibrated	2.62	5.40	11.05	1.24	37%
$c \cdot d_k$ recalibrated	1.96	3.48	4.38	1.21	28%

Table 7: Efficiency relative to the oracle on the density, sixteen densities, twelve samples each, $n = 500$. Lower is better and 1 is unattainable.

LSCV wins on every column, and it is the only selector without a catastrophic density anywhere in the benchmark. That last property is the one to weigh. The benchmark averages sixteen densities, but the method will be run once, on one sample from one distribution, so a selector that is excellent on average and disastrous on one shape is a poor bet. Every choice in this report is scored the same way from here on: on its worst case over the benchmark rather than on its mean.

The normal-reference rules perform worse here than their reputation suggests. Silverman, Scott and the plug-in are built on the assumption that the target resembles a normal density, and they oversmooth multimodal shapes systematically; on the mixture of five separated modes their efficiencies are 21, 23 and 19. They are the best selectors on the two smooth unimodal densities of the benchmark, where LSCV sits at 1.5 to 1.6 and they sit between 1.1 and 1.4. If the target were known to be smooth they would be the right choice, which is not our situation.

Repeating the study at $n = 1000$ leaves the ranking unchanged and sharpens two features. LSCV improves, to a mean of 1.20 and 3% of cases above twice the oracle, which is what a consistent selector should do. The normal-reference rules get worse: Silverman goes from 3.77 to 5.42 in the mean and from 21 to 35 in the worst case. That is not a paradox. Their error is a systematic smoothing bias that does not shrink with n, while the oracle they are compared against keeps improving, so the ratio grows. It is the signature of a model error rather than a variance one.

Cross-validation is unstable, and by a wide margin. Across the sixteen densities the coefficient of variation of h over twelve samples ranges from 0.10 to 0.35 for LSCV, against 0.02 to 0.16 for the plug-in and 0.01 to 0.17 for Silverman. LSCV is indeed the noisiest of the three.

That noise does not reach the error. LSCV's 90th percentile efficiency is 1.61 and its worst case 1.59, so there is no tail: the window wobbles by roughly a quarter between samples and the resulting estimate barely moves. The reason is that the error curve is flat near its minimum. Missing the optimal window by 25% costs almost nothing, while missing it by a factor of three, which is what a fixed rule does on a density that does not resemble its calibration set, costs

everything. The stability of a selector has to be judged on the error it produces, not on the steadiness of its output: Scott's rule is almost perfectly stable and almost always wrong.

4.6 What the square-root-of-n schedule costs

The schedule $h_n = h_1/\sqrt{n}$ satisfies the consistency conditions, but it is not the rate that minimises the error. For density estimation the asymptotically optimal window shrinks like $n^{-1/5}$, so fixing the exponent at $1/2$ forces the discrepancy onto h_1. Two questions follow: how far off is the exponent in practice, and what does the mismatch cost.

Measuring the oracle window across budgets from 100 to 2000 and regressing its logarithm on $\log n$ gives an exponent of 0.299 on average, ranging from 0.232 on the Gaussian to 0.388 on the asymmetric claw. The empirical rate therefore sits between the theoretical 0.2 and the schedule's 0.5, closer to the former, and it drifts upward on densities with fine structure, which is consistent with those still being in a pre-asymptotic regime at these sample sizes.

The cost is easier to state than the rate. Taking h_1 from the oracle at one budget and using it at another, through $h = h_1/\sqrt{n}$, gives efficiencies of 1.00 to 1.11 when the two budgets coincide, at most about 1.30 when they differ by one step, and up to 5.39 when they differ by a factor of twenty. At a fixed sample size the schedule costs nothing, since any window whatsoever can be written as $h_1/\sqrt{n}$ for a suitable h_1. Moving between 500 and 1000 costs a few percent. What is not legitimate is treating a value of h_1 calibrated at one budget as a constant of the problem.

4.7 What is delivered

The window is chosen by least-squares cross-validation on a grid of candidates built from a robust scale estimate, and reported in the form the schedule requires, as $\hat{h}_1 = \hat{h}\sqrt{n}$.

This needs stating precisely, because it is the point on which the procedure is most likely to be questioned. We are not computing a window by one route and then dressing it up as h_1 afterwards. We are *estimating* h_1, which was never a universal constant to begin with: $1.5\,\hat{\sigma}$ is itself a function of the sample, since $\hat{\sigma}$ is computed from the data. Both quantities have the same formal status, and what changes is the quality of the estimator, which is what Table 7 measures.

The estimated $\hat{h}_1$ does depend on the sample, and it is not constant across budgets either, for the reason given in Section 4.6. Both properties are shared with the rule it replaces. What is not shared is the ratio $\hat{h}_1/\hat{\sigma}$, which a fixed rule holds constant and which in fact ranges from 0.32 to 3.77 across the sixteen densities of the benchmark at $n = 500$. That spread is produced by the recommended selector running on data, not by an oracle: even a procedure that looks at the sample does not find a constant ratio, because there is no constant ratio to find.

5 The network: enforcing the shape of a distribution function

5.1 What the output must guarantee

The network is asked to produce a distribution function, which is a more constrained object than a generic curve. Four properties are needed, and they are not equally easy to obtain.

The output must lie in $[0, 1]$, must be non-decreasing, and must approach 0 and 1 at the two ends of the real line. It must also be differentiable, and here the requirement is sharper than it looks: the density is obtained by differentiating the network, so the smoothness of the output is the smoothness of the delivered density. A network with piecewise-linear activations such as ReLU would produce a piecewise-constant density, which is not merely inelegant but wrong for the purpose. This is why the activations used throughout are smooth ones.

The first three properties are constraints on shape, and the interesting question is *where* they are enforced: inside the model, so that they hold for any parameter values, or afterwards, by repairing whatever the model produced.

5.2 The first attempt and its limits

The first architecture one would reach for, and the one we started from, is a small multilayer perceptron with a sigmoid on the output,

$$F_\theta(x) \;=\; \sigma\Big(\textstyle\sum_j w_j\,\sigma(a_j x + b_j) + c\Big), \tag{5}$$

with the final sigmoid supplying the range constraint. Monotonicity was not enforced at all during training; it was repaired afterwards, by evaluating the network on a grid, taking a running maximum so that the curve never decreases, and rescaling the result to span $[0, 1]$ exactly. The density was then obtained as a finite difference of that repaired curve.

That arrangement works, in the sense that what comes out is a monotone curve from 0 to 1 with unit mass. What it actually guarantees, however, is three things of rather different standing.

The range is genuinely enforced by the final sigmoid. Monotonicity is enforced only after the fact, and only at the grid points that were evaluated. The limits at 0 and 1 are not enforced at all: they are imposed by the rescaling, which stretches whatever the network produced so that it starts at 0 and ends at 1 at the two ends of the grid. Since the grid in a study of this kind is naturally built from the known distribution, that last step uses information which does not exist when the method is applied to real data.

There is also a subtler consequence. The delivered density is a finite difference of a post-processed curve, not the derivative of the network, so its accuracy depends on the grid spacing, and the object handed to the user is a table rather than a function.

5.3 The saturation theorem

The reason the limits cannot be enforced inside architecture (5) is structural rather than a matter of training.

Theorem 1 (Saturation)**.** *Let $F_\theta(x) = \sigma(u(x))$ where u is a finite sum of bounded activations with finite parameters. Then u is bounded, and consequently*

$$\sigma(m) \;\le\; F_\theta(x) \;\le\; \sigma(M) \qquad \textit{for all } x,$$

with m and M finite. Both bounds are strictly inside $(0, 1)$, so the values 0 and 1 are never attained at any finite parameter setting, and are approached only as $\|w\|$ diverges.

The proof is in Appendix A; it is three lines and amounts to the observation that a finite sum of bounded terms is bounded. The consequence is that reaching the tails requires the weights to diverge, so gradient descent can only creep towards them.

The effect is measurable. Training architecture (5) on the reference trimodal mixture at $n = 1000$ and evaluating it fifty standard deviations away from the data, the total mass it can span is 0.9955 in the unconstrained version and 0.9903 in the positive-weight one. Around half a percent of the probability is unreachable, and the rescaling step does not recover it: it redistributes that deficit back into the interior of the domain, converting a tail error into an interior one.

5.4 The chosen architecture

The alternative is to build the constraints into the functional form. Write

$$F_\theta(x) \;=\; \sum_{j=1}^{J} \pi_j \, \sigma(a_j z + b_j)\,, \qquad z = \frac{x-\mu}{s}, \qquad \pi = \mathrm{softmax}(u), \qquad a_j = \mathrm{softplus}(\alpha_j), \quad (6)$$

with μ and s the median and standard deviation of the sample. The free parameters are α, b and u, each of length J, and the two reparameterisations do all the work: the softmax forces the weights π_j to be positive and to sum to one, and the softplus forces the slopes a_j to be positive.

The form is not new. It is the transformation used in deep sigmoidal flows (Huang et al., 2018) without the logit that those apply to the output, which is what keeps our version inside $[0,1]$ instead of mapping back to the whole line.

What makes it a natural choice here is its relationship with the estimator of Chapter 3. Equation (3) with the logistic kernel is an average of n sigmoids, one centred on each observation, all sharing the same width h. Equation (6) is a weighted sum of J sigmoids, with the weights, the centres $-b_j/a_j$ and the widths $1/a_j$ all learned. The network is therefore the Parzen estimator with n replaced by J, and with the weights and widths freed rather than fixed. With $J \ll n$ it is a compression of the estimator it is trained on, which is a more honest description of what the neural stage contributes than any claim of superior accuracy.

5.5 The guarantees, proved

Theorem 2 (Validity by construction)**.** *Let F_θ be given by (6) with $\pi_j > 0$, $\sum_j \pi_j = 1$ and $a_j > 0$. Then F_θ is the distribution function of an absolutely continuous distribution on $\mathbb{R}$: it is infinitely differentiable, strictly increasing, $F_\theta(-\infty) = 0$ and $F_\theta(+\infty) = 1$ exactly, and its derivative is strictly positive with integral one.*

The proof is in Appendix A. The property that matters in practice is not the theorem itself but its scope: because softmax and softplus map the whole of $\mathbb{R}^{3J}$ into the admissible region, the hypotheses hold at *every* parameter value. There is no feasible set to stay inside, no constraint for the optimiser to violate, and no repair step that could be forgotten.

The guarantees survive finite-precision arithmetic, which is not automatic. Drawing a thousand random parameter vectors with large perturbations and checking monotonicity on five thousand points, with a tolerance of 10^{-6} appropriate to single precision, the unconstrained architecture (5) is non-monotone 900 times out of 1000; both the positive-weight variant and the mixture are non-monotone in none. Evaluated at $\pm 10^6$ the mixture returns exactly 0 and exactly 1, and its density integrates to 1.000000 over a wide domain without any normalisation.

The comparison with the positive-weight variant should be made explicitly, since on monotonicity alone the two are equivalent. What separates them is everything else: the positive-weight network still cannot reach 0 and 1 by Theorem 1, still requires a grid to produce a density, and is not equivariant under rescaling of the data.

5.6 No loss of expressiveness

Constraining a model usually costs something, so it is fair to ask what the restriction in (6) rules out.

The first answer is exact. Setting $J = n$, $\pi_j = 1/n$, $a_j = 1/h$ and $b_j = -x_j/h$ recovers the Parzen estimator (3) identically. The family therefore contains the very estimator the network is trained on, so nothing is lost by construction; whatever is given up by taking $J \ll n$ is the compression we are choosing, not a limitation of the form.

The second is asymptotic. For any continuous distribution function G and any J, placing the centres at the quantiles $G^{-1}((j-\tfrac{1}{2})/J)$ with equal weights and letting the widths shrink

gives $\sup_x |F_\theta(x) - G(x)| \le 1/J$, so the family is dense in the supremum norm as J grows. The argument is in Appendix A. Universality for the same family, in the flow formulation, is also established in the literature cited above.

5.7 Standardisation and equivariance

An estimator of a distribution should not care where the data sits on the axis. If every observation is multiplied by α and shifted by β, the estimate should transform along with it and otherwise be identical.

The Parzen estimator has this property already, provided the window is chosen equivariantly: $\hat\sigma$ scales with α, so h does too, and the kernel arguments $(x - x_i)/h$ are unchanged. A network does not get it for free. In architecture (5) the weights are initialised without reference to the data and the biases start at zero, so every sigmoid is centred at the origin regardless of where the sample lies, and the optimiser works in absolute units.

The consequence is severe and easy to miss, because the mixtures one tests on conventionally all sit near the origin. Translating the same trimodal sample by 100 and retraining, that architecture returns a Kolmogorov–Smirnov distance of 0.50 on two samples out of three and 0.70 on the third, against 0.03 untranslated. A KS distance of 0.5 means the network has collapsed to the constant 0.5: it has learned nothing at all.

The fix is the standardisation already written into (6): the model works on $z = (x-\mu)/s$ and stores μ and s, so that a shift or a rescaling of the data leaves the internal problem untouched.

Theorem 3 (Equivariance)**.** *If μ and s are equivariant statistics, then the estimate computed on $\alpha x + \beta$ evaluated at $\alpha t + \beta$ equals the estimate computed on x evaluated at t, for every $\alpha > 0$ and β.*

In exact arithmetic the two estimates coincide. In single precision they do not quite, and what costs the precision is the subtraction $x - \mu$ itself. As the sample moves away from the origin the two terms become close in magnitude and their difference loses relative precision, so the standardised sample is reproduced to 2×10^{-7} at the origin, 1.5×10^{-5} at an offset of a thousand, and 2.4×10^{-4} at ten thousand.

What survives of that in the delivered estimate is smaller still. Measured on three samples, the Kolmogorov–Smirnov distance changes by at most 2×10^{-4} across a translation of ten thousand and a rescaling by fifty, against a collapse to 0.50 without standardisation. The practical limit is therefore set by single-precision arithmetic rather than by the construction, and it sits three orders of magnitude below the estimation error itself.

5.8 How many components

The number of components J is the only capacity knob, and it behaves the way capacity usually does, with the twist that both directions of error are visible on this benchmark. Below the number of modes of the target the model simply cannot represent it; above it, the extra freedom is spent fitting the noise in the labels.

Target	$J = 4$	$J = 6$	$J = 8$	$J = 12$	$J = 24$
trimodal	**0.0027**	0.0029	0.0034	0.0044	0.0049
five narrow separated modes	0.0536	0.0047	0.0047	**0.0047**	0.0047
six modes at mixed scales	0.0343	0.0183	0.0049	**0.0025**	0.0026
worst case	0.0536	0.0183	0.0049	**0.0047**	0.0049

Table 8: Integrated squared error on the density at $n = 500$, averaged over three samples.

The choice falls on the worst row. The risk is also strongly asymmetric: choosing J too small is catastrophic, since $J = 4$ costs a factor of eleven on the five-mode target, while choosing it too large costs a few percent. We use $J = 12$.

6 Training

6.1 The labels: the Parzen Neural Network recipe

The network is trained by regression against the Parzen estimator, at the sample points and nowhere else. For each observation x_i the label is the value the estimator takes there, computed *without* the contribution of x_i itself:

$$y_i \;=\; \frac{n\,\widehat{F}(x_i) - \frac{1}{2}}{n-1}. \tag{7}$$

The correction is exact rather than approximate. In the full estimate (3) the term contributed by x_i is $K(0) = \frac{1}{2}$ for any symmetric kernel, so subtracting it and averaging over the remaining $n - 1$ observations gives the leave-one-out value in closed form, with no need to recompute anything.

Training only at the sample points, rather than on a dense set of synthetic inputs, is part of the recipe rather than an economy. The estimator is trustworthy where there is data and increasingly speculative elsewhere; asking the network to match it in the empty regions would be fitting an extrapolation.

6.2 Why a deliberately narrow window

The window used to build the labels is not the one that minimises the error of the Parzen estimator. It is narrower, by a factor of two in our implementation.

The reasoning is the one behind the Parzen Neural Network construction (Trentin). A narrow window produces labels that are noisy but close to unbiased: each y_i fluctuates around the true value of F at that point rather than around a smoothed version of it. A network with limited capacity cannot follow those fluctuations, so it averages them out, and what it converges to is closer to the underlying curve than the labels it was shown. Widening the window would remove the noise before training, but it would also remove the signal it is hiding, and the network cannot recover what the smoothing discarded.

How narrow is a separate question from whether, and the factor of two is a round number, so we swept it.

Target	$0.25\,\hat{h}$	$0.5\,\hat{h}$	$0.75\,\hat{h}$	$\hat{h}$	$1.5\,\hat{h}$
trimodal	0.00509	0.00419	0.00352	**0.00330**	0.00425
five narrow separated modes	0.00398	**0.00346**	0.00400	0.00612	0.01437
six modes at mixed scales	0.00326	0.00289	**0.00281**	0.00316	0.00505
worst case	0.00509	0.00419	**0.00400**	0.00612	0.01437
relative to best	1.27	1.05	**1.00**	1.53	3.59

Table 9: Integrated squared error on the density at $n = 500$, averaged over five samples, as the window used for the labels varies. The last column widens the teacher beyond the estimator's own window rather than narrowing it.

The direction is confirmed and the cost of getting it wrong is asymmetric. Using the estimator's own window for the labels costs 1.53 on the worst case, and widening it by half again

costs 3.59. The damage falls on the five narrow modes, where over-smoothed labels erase the structure the network is there to recover, and the error grows by a factor of four.

The factor itself matters much less. Between 0.5 and 0.75 the worst case moves by five per cent, and the three targets disagree about which is best, putting their individual minima at 1.00, 0.5 and 0.75 respectively. That disagreement is the reason not to calibrate the factor finely: it would be fitting the benchmark rather than the mechanism. We keep the half, which sits inside the flat region and is the value every other measurement in this report was taken with.

6.3 What is no longer needed

A design built on architecture (5) needs three further mechanisms to deliver a valid estimate. None of them has any object under the architecture of Chapter 5, and dropping them is not a simplification for its own sake.

The **monotonicity penalty** added a term $\lambda \cdot \operatorname{mean}\operatorname{relu}(-\widehat{F}')$ to the loss, evaluated at 256 equally spaced points. It is a penalty and not a constraint: it can be made small but never zero-guaranteed, it certifies only the points it looks at, and λ trades against the fit. Under Theorem 2 there is nothing left for it to enforce.

The **rectification** step, the running maximum followed by rescaling, does real work, and that should be acknowledged: an unconstrained network genuinely produces non-monotone curves. Training architecture (5) on these labels, about one initialisation in four yields a curve that decreases somewhere, in the worst case over 6% of the grid points. The step was therefore not decorative. It is simply the wrong remedy: a repair applied to the output cannot restore a property the model never had, and it introduces its own artefacts, since a flat stretch in the repaired curve means a density of exactly zero there.

The **clamp** that set negative densities to zero is the one that did damage. Zeroing the negative part of $\widehat{f}$ adds area, so the identity

$$\int_a^b \widehat{f} \;=\; \widehat{F}(b) - \widehat{F}(a)$$

no longer holds, and the two objects handed to the user stop being derivative and primitive of one another. On a network that genuinely violates monotonicity the clamp activates on between 32 and 132 grid points and inflates the mass by 5×10^{-6} to 10^{-4}. The magnitude is small; the point is that it is a systematic error in a fixed direction, introduced by a step whose purpose was to make the output look valid.

6.4 Reproducibility

Full-batch training with a fixed number of steps is deterministic, so the only randomness in the neural stage is the initialisation of the weights. That makes reproducibility a question of when the seed is applied, and it is easy to get wrong: applying a seed after the model has been constructed has no effect at all, since the weights have already been drawn.

We check this rather than assume it, with two properties that a correctly seeded procedure must satisfy: the same seed must give the same result regardless of what else has consumed random numbers beforehand, and different seeds must give different results. Both are asserted in the test suite.

The architecture of Chapter 5 satisfies something stronger, and by construction rather than by discipline. Its initialisation places the component centres at the quantiles of the standardised sample and gives every component equal weight, which consumes no random numbers at all. The trained model is therefore a deterministic function of the data, and the seed is irrelevant rather than merely respected. A practical consequence is that averaging over repetitions now measures what it should: the variability across repetitions comes from the samples alone, not from the accident of where the optimiser started.

7 From the distribution function to the density

7.1 The closed-form derivative

The density is the derivative of (6), and it can be written down:

$$\widehat{f}(x) \;=\; \frac{1}{s}\sum_{j=1}^{J} \pi_j \, a_j \, \sigma(z_j)\,\bigl(1-\sigma(z_j)\bigr), \qquad z_j = a_j\frac{x-\mu}{s} + b_j. \tag{8}$$

Every factor is positive, so the density is positive without any clamping; the sum of the π_j is one, so it integrates to one without any normalisation; and it is the exact derivative of the delivered distribution function, not an approximation to it, so the two outputs are consistent by algebra rather than by numerical accident.

The final stage of the pipeline is therefore a formula rather than a procedure, with no grid to choose and no differentiation step to tune.

7.2 Why finite differences on a grid were a problem

The alternative, which also produces a plausible-looking density, deserves a concrete comparison.

There, the density is obtained by evaluating the repaired curve at the nodes of a grid and taking centred differences between neighbouring values. Three things follow from that. The result depends on the grid spacing, which nobody had calibrated: too coarse and narrow peaks are flattened, too fine and the differencing amplifies whatever wiggle the curve has. The output is a table of numbers rather than a function, so evaluating the density at a point that is not a node requires interpolating, and evaluating it outside the grid is impossible. And the clamp discussed in Section 6.3 was applied to the result, breaking the relationship between the two delivered curves.

None of it is visible in a plot, because the plot is drawn on the same grid that produced the estimate.

8 Delivering an estimate without knowing the truth

8.1 The evaluation domain

The estimate is a function and needs no domain. Everything one does *with* it does: a plot needs an interval, an integral needs limits, and any diagnostic that involves the mass has to say over what. The question is where that interval comes from when the distribution is unknown.

In a study where the distribution is known there is an obvious answer, which is to take the interval spanned by the component means plus or minus a few component standard deviations, and that is what we did while the targets were all synthetic. On the data the method is meant for there are no component means, so the interval has to be built from the sample. The obvious candidate is the range of the data padded by some multiple of the window,

$$[\,\min x - kh,\ \max x + kh\,],$$

and it turns out to be a poor one, for a reason that took a measurement to see.

The window is chosen to resolve the finest structure in the density, so it is *small* precisely on the distributions whose tails are wide: a narrow spike sitting inside a broad component pulls h down, while the broad component's tails still extend over a distance of order σ. Padding by a multiple of h is then negligible against what needs covering. On the kurtotic and outlier densities of the benchmark the mass left outside barely improves between $k = 0$ and $k = 10$.

Anchoring the padding to the dispersion instead works, and works with a comfortable margin:

Rule	Mean	Worst case
data range only	3.2×10^{-3}	4.6×10^{-3}
padded by $3h$	1.2×10^{-3}	3.0×10^{-3}
padded by $10h$	3.9×10^{-4}	1.7×10^{-3}
padded by $3\hat{\sigma}$	$\mathbf{7.0 \times 10^{-6}}$	$\mathbf{1.1 \times 10^{-4}}$
padded by $3\times$ robust scale	5.4×10^{-5}	7.9×10^{-4}
quantiles of the estimated CDF	8.6×10^{-4}	2.8×10^{-3}

Table 10: True probability mass left outside the evaluation domain, sixteen densities, eight samples each, $n = 500$.

Two rows are instructive beyond the winner. Using the data range alone leaves about $2/(n+1)$ outside, which is what order statistics dictate and is independent of the density; that is the floor any padding has to improve on. And the rule that looked most elegant, using the quantiles of the estimated distribution function to decide where to stop, performs badly for the same reason as the h-padded rules: the tail of $\widehat{F}$ decays at a rate set by h, so its quantiles stay close to the data.

The robust scale, which is the right choice for selecting the window, is the wrong one here: it is designed to *under*-estimate dispersion in the presence of heavy tails, which is useful when deciding resolution and harmful when deciding extent. The general principle is that the dispersion of the sample sets how far to go, while the window sets how finely to sample in between, and the two jobs need different quantities.

We use $[\min x - 3\hat{\sigma},\ \max x + 3\hat{\sigma}]$. Going wider is not free, since the number of grid nodes is finite and widening spends resolution where there is nothing to see.

8.2 Diagnostics computable from the samples alone

On the data this method is built for there is no true curve to compare against, so the usual quality figures are unavailable. What remains has to be computed from the sample, and the question is which of them are actually informative.

We measured it directly. For each density, sweeping the window over a wide range, we recorded the true integrated squared error alongside four candidate indicators, and asked whether each indicator orders the windows the way the truth does. Rank correlation answers that question:

Indicator	Rank corr. with true error	Cost if used to pick h
LSCV score	$+0.94$	1.21
leave-one-out log-likelihood	$+0.86$	1.52
mass over the domain	$+0.44$	5.95
KS against the empirical CDF	-0.03	13.66

Table 11: Sixteen densities, eight samples each, twenty-five windows per sample. The last column is the efficiency of the window that minimises each indicator.

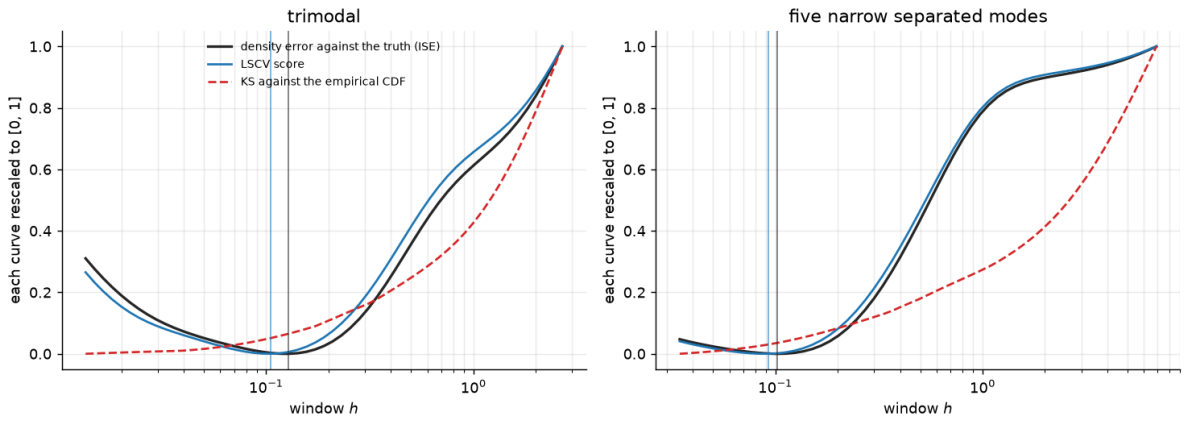

Figure 1: What each indicator says about the window, on the reference trimodal (left) and on five narrow separated modes (right), at $n = 500$. Each curve is rescaled to $[0, 1]$ separately, because what matters is where it falls, not what it is worth. The cross-validated score follows the true density error closely and its minimum sits almost on top of it: taking the window it selects costs 6% of the achievable error on the left and 3% on the right. The comparison against the empirical distribution function does something else entirely. It decreases without a minimum as the window shrinks, so followed to its conclusion it selects the smallest window on offer, where the density error is 8.3 and 2.5 times the achievable one. The vertical lines mark the two minima that exist.

The last row is the indicator a reader would most expect to see. Comparing the estimated distribution function against the empirical one, and reporting how closely they agree, looks like the natural way to demonstrate that an estimate fits its data. It is anticorrelated with the actual error, and choosing a window by it costs a factor of fourteen on average and ninety-five in the worst case.

The reason is structural. As $h \to 0$ the Parzen estimate converges to the empirical distribution function by construction, so that comparison rewards the narrowest window available, which is the most overfitted one. It measures how closely the estimate reproduces the sample, and reproducing the sample is precisely what a density estimate should not do. Figure 1 shows the three curves side by side: the cross-validated score turns where the true error turns, while the comparison against the empirical distribution function never turns at all. Both the figure and the interactive tool display that quantity, labelled as the trap it is, because an indicator that fails this way is more useful watched than described. Neither uses it to select anything.

What we report instead is the LSCV score and the leave-one-out log-likelihood, which rank windows almost as the truth would, together with the selected window in both forms, the mass over the domain and the number of monotonicity violations. The last two are diagnostics rather than results: under Theorem 2 the violations must be zero, so a non-zero count is a bug and not a phenomenon. The mass is *reported and not imposed*, which is the substantive difference from a pipeline that rescales. A value of 0.9998 says something true about the estimate and the domain; a value of exactly 1 obtained by rescaling says nothing.

A further diagnostic comes for free. Silverman's rule and the plug-in are cheap to evaluate even when they are not used, and the ratio between them and the selected window is informative: a discrepancy beyond a factor of three indicates a target with structure at several scales, since that is exactly the situation in which the normal-reference rules oversmooth. On the reference trimodal mixture that ratio is about six, and the implementation raises it as a warning rather than hiding it.

8.3 The interface

The whole procedure is reached through one function, which takes a vector of observations and returns an object exposing $\widehat{F}$, $\widehat{f}$, the selected window in both forms, the evaluation domain and the diagnostics. It imports nothing that describes a distribution; a test in the suite parses the module and fails if it ever does.

For use from outside there is a command line:

```
python -m parzen_cdf.run -samples data.csv -out results/
```

which reads a vector of numbers from a `.csv`, `.txt` or `.npy` file and writes the two estimated curves tabulated over the domain, the diagnostics as JSON, the fitted model, and a figure. Nothing in that path requires knowing where the data came from.

A file written by someone else can be read in more than one way, and the two readings give different samples. A column of values like `1,5` is either one number with a decimal comma or two columns; a file of pairs is either observations or an index beside them. Both readings produce numbers, and an estimate computed on the wrong one looks entirely ordinary, so the reader refuses the ambiguous cases and names the option that resolves them instead of choosing on the user's behalf.

9 Results

9.1 Against a conventional design

Everything up to here argued for individual choices. What follows measures the procedure as a whole against the conventional alternative: the multilayer perceptron of Section 5.2, trained on the same labels, with monotonicity restored by a running maximum and the density taken as a finite difference. That is the design one reaches without the analysis of Chapters 5 and 6, so it is the right thing to beat.

Both are given the same samples and the same evaluation domain, built from the data by the rule of Section 8.1. The second point is not a detail: a domain taken from the true distribution would have made the baseline look worse for a reason unrelated to its architecture, and the comparison would have been rigged in our favour.

Target	Pipeline	KS	ISE	Mass	Viol.
trimodal	Parzen (LSCV)	0.0346	0.00398	1.000000	0
	MLP baseline	0.0365	**0.00340**	1.000000	0
	present	0.0348	0.00419	1.000000	0
symmetric bimodal	Parzen (LSCV)	0.0393	0.00304	1.000000	0
	MLP baseline	0.0397	0.00318	1.000000	0
	present	**0.0390**	**0.00304**	1.000000	0
spike inside a broad mode	Parzen (LSCV)	0.0331	0.00771	1.000000	0
	MLP baseline	0.0339	**0.00620**	1.000000	0
	present	**0.0321**	0.00649	0.999990	0
five narrow separated modes	Parzen (LSCV)	0.0393	0.00702	1.000000	0
	MLP baseline	0.0396	0.00667	1.000000	0
	present	**0.0364**	**0.00346**	1.000000	0
six modes at mixed scales	Parzen (LSCV)	0.0389	0.00396	1.000000	0
	MLP baseline	0.0592	0.01026	1.000000	0
	present	**0.0385**	**0.00289**	0.999998	0
all targets	Parzen (LSCV)	0.0370	0.00514	1.000000	0
	MLP baseline	0.0418	0.00594	1.000000	0
	present	**0.0361**	**0.00402**	0.999997	0

Table 12: Five distributions, five samples each, $n = 500$. The last block averages over all of them; the worst case across targets is 0.0390 for the present pipeline against 0.0592 for the baseline on the KS distance, and 0.00649 against 0.01026 on the density.

The procedure is ahead on both averages and, more usefully, on both worst cases. The margin is concentrated where it should be: on the six-mode target at mixed scales, which is the closest thing in this set to the strongly multimodal case, it improves the KS distance by a factor of 1.5 and the density error by 3.5. On the easier targets the difference is within the noise, and on the trimodal the baseline is slightly better on the density.

The mass column carries more than it appears to. The baseline reports exactly 1.000000 on every row, which is the rescaling step doing what it was written to do rather than a property of the estimate. The procedure reports 0.999990 and 0.999998 on two rows, because that is the mass it actually places on a finite domain, and the shortfall of about 10^{-5} is the tail beyond the interval. That quantity is not absent from the baseline's figures; it is redistributed into the interior by the rescaling, where it becomes an error that nothing reports.

Monotonicity violations are zero everywhere for both, but for different reasons: in one case because a repair step removed them after the fact, in the other because Theorem 2 does not allow them to exist.

9.2 Against the Parzen estimator

The comparison that matters more for the assignment is against the estimator the network learns from, since a neural stage that does not improve on its teacher would be hard to justify.

Averaged over the five targets, the network is ahead on both measures: 0.0361 against 0.0370 on the KS distance and 0.00402 against 0.00514 on the density. The gain falls on the density, which is where the mechanism of Section 6.2 predicts it. On the distribution function the two are nearly tied, which is equally expected, since integration already suppresses most of the fluctuation the network is there to remove.

The advantage is real but should not be overstated, and it is not uniform. On the resilience

study of Chapter 10 there are targets where the network loses to the estimator, and one where it loses by a factor of two. What the neural stage reliably provides is form rather than accuracy. It returns a function instead of a table, valid by construction, with a density in closed form and a cost of evaluation independent of n.

9.3 What the figures show

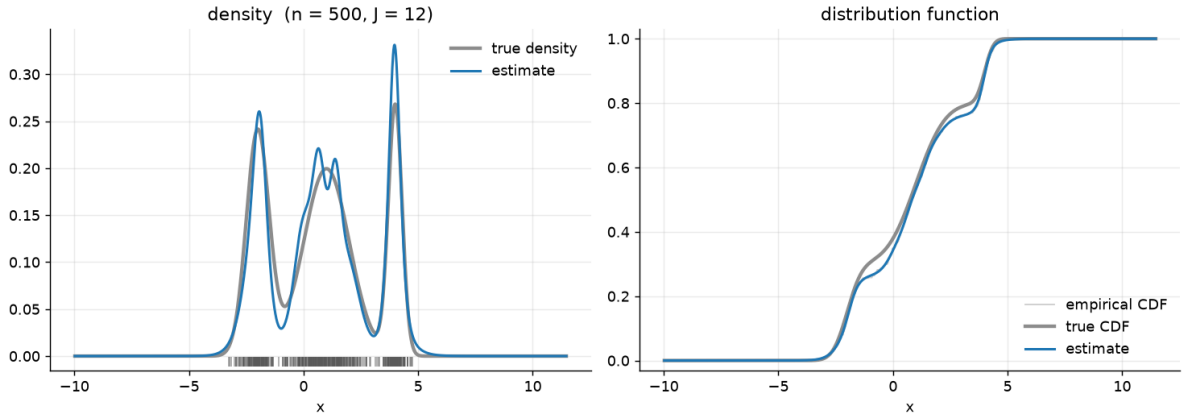

Figure 2: The delivered output on five hundred points from the reference trimodal mixture. Tick marks along the axis are the observations. On the right, the empirical distribution function is drawn underneath and is almost indistinguishable from the other two at this scale.

Figure 2 is the output of the procedure on a single sample, produced by the same function a user would call. All three modes are recovered, including the narrow one on the right, and the tails settle onto 0 and 1 rather than approaching them.

Two artefacts are visible. The central mode is split into a double bump that the truth does not have, and the right-hand peak overshoots by about a quarter. Both come from the same source: the labels are built with a deliberately narrow window, so they carry the sampling fluctuation of this particular sample, and twelve components are enough to follow part of it. Widening the label window would smooth the bump away and flatten the genuine peaks with it. This is the trade discussed in Section 6.2, made visible.

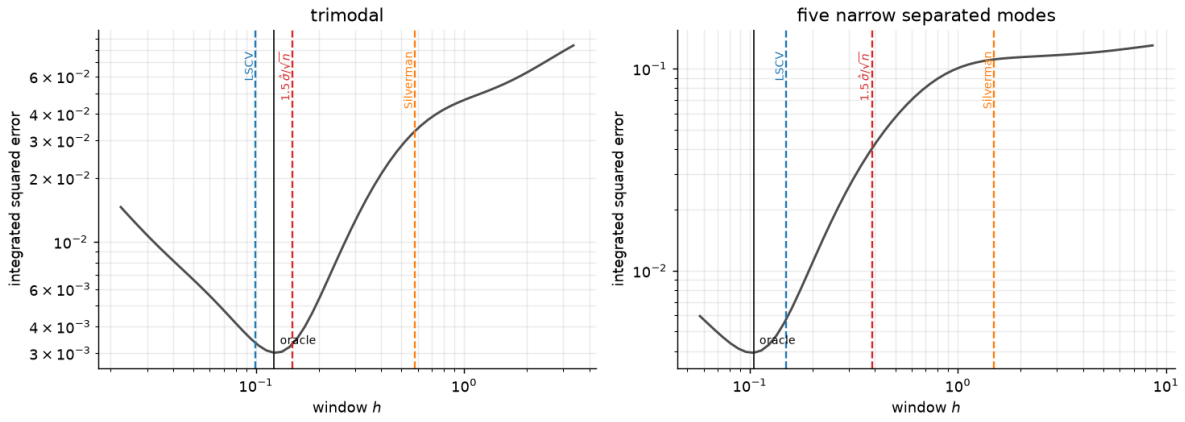

Figure 3: Density error against window width, on two targets, with the choices made by three selectors marked. Both axes are logarithmic. The vertical black line is the best window achievable on that sample.

Figure 3 is the argument of Chapter 4 in one picture. The error curve is markedly asymmetric: to the left of the minimum it rises gently, to the right it climbs by more than an order of magnitude. Underestimating the window is cheap and overestimating it is not, which is why a rule calibrated on smooth targets is dangerous on structured ones.

The marked positions show what that costs in practice. On the trimodal target, on the left, all three selectors land in the neighbourhood of the minimum, and the differences between them barely matter. On the five separated modes, on the right, they separate: cross-validation stays close to the optimum, the σ-rule sits an order of magnitude up the steep branch, and Silverman's rule lands where the error is roughly thirty times the achievable one. The mechanism is the one described earlier: the sample standard deviation is inflated by the distance between the modes, so any rule proportional to it oversmooths exactly when the target has separated structure.

10 Scope and limits

10.1 Distributions from other families

Every calibration in this report was carried out on mixtures of normals: the window, the number of components, the rule for the domain. The assignment concerns a multimodal distribution, which makes that family a reasonable working assumption, but it is an assumption, and it is cheap to test now that the procedure accepts any vector of numbers.

We applied it unchanged to twelve distributions from other families: uniform, exponential, lognormal, Student's t with three, two and one degrees of freedom, and mixtures combining these with normals. The t with two degrees of freedom has infinite variance and the Cauchy has no mean, so they probe the assumptions harder than anything in the main benchmark.

Before measuring we wrote down what we expected to break. The domain rule uses $\hat{\sigma}$, which is unstable under heavy tails and undefined in the limit, so we expected the domain to inflate. The logistic kernel has exponential tails, so we expected polynomial ones to be poorly reproduced. And a mixture of logistic distribution functions has strictly positive density everywhere, so we expected sharp edges to be rounded off.

The first two expectations were wrong.

The domain tightens instead of inflating: the ratio between its width and the range of the data falls from 2.64 on the Gaussian reference to 1.27 on the Cauchy. The sample standard deviation does grow on heavy tails, but the range $\max x - \min x$ grows faster, because it is governed by extreme order statistics, so the additive margin of $3\hat{\sigma}$ becomes negligible against an already very wide interval. The rule corrects itself for a reason we had not anticipated.

Heavy tails turned out to be easier. The t with three degrees of freedom gives an integrated squared error of 0.4 times the Gaussian reference, and the network beats the Parzen estimator on it by a factor of 0.7. The argument we had made confused two things: the tails are indeed reproduced badly, but they contain very little probability, and the integrated squared error is dominated by the central region. By that measure the difficulty of a density lies in its fine structure, not in the weight of its tails, and a t density is a single smooth peak.

10.2 The structural limit

The third expectation was correct, and it is the one real limitation of the method.

Target	ISE	vs. Gaussian ref.	Network / Parzen
Gaussian mixture (reference)	0.0038	1.0×	0.87
Student's t, 3 d.o.f.	0.0016	0.4×	0.70
lognormal	0.0057	1.5×	1.17
normal + uniform	0.0117	3.1×	1.22
uniform	0.0220	5.8×	1.26
exponential	0.0303	8.0×	1.24
two separated uniforms	0.0521	13.8×	2.20

Table 13: Density error on distributions from other families, $n = 500$, five samples each. The last column is the ratio between the network and the Parzen estimator it learns from; above 1 the network is worse.

A discontinuity in the density cannot be represented by a finite mixture of smooth distribution functions, and the error grows accordingly: nearly six times the reference on a uniform, eight on an exponential, fourteen on two separated uniforms. The last case is also the only one in the whole study where the network is clearly worse than its own teacher, by a factor of two, which fits the explanation: the Parzen estimator has n kernels available to approximate the jump and the network has twelve components.

That suggests a remedy, and we tested it rather than assuming it would work. Raising the number of components from twelve to forty-eight halves the error on two separated uniforms, from 0.054 to 0.027, and worsens it by 39% on the trimodal Gaussian mixture and by 31% on the six-mode one. The trade runs in both directions, so there is no free improvement to collect, and even at forty-eight components the network still does not catch the Parzen estimator on the edges. We keep twelve components, which is calibrated for the targets the assignment describes. On a distribution with compact support the estimate remains a valid distribution function, but its density rounds off the edges and the error grows by a factor between three and fourteen.

10.3 What we do not claim

Several statements that would be convenient are not supported by what we measured, and it seems more useful to list them than to leave them implicit.

We do not claim that the network is more accurate than the Parzen estimator in general. It is ahead on average over the five main targets, and Table 13 contains targets where it is behind, one of them by a factor of two. What it delivers reliably is the form described in Section 9.2, not a uniform gain in accuracy.

Nor is cross-validation the best window selector in all circumstances: on smooth unimodal targets the plug-in and Scott's rule beat it. The recommendation is conditional on the situation this project is in, which is an unknown and declaredly multimodal target.

The padding of $3\hat{\sigma}$ for the domain is sufficient rather than optimal. Over sixteen densities the worst case leaves a mass of 1.1×10^{-4} outside, two orders of magnitude below the typical estimation error. On a distribution with much heavier tails than those tested it might not be, and we have not tested one.

The rank correlations of Table 11 calibrate nothing. A high rank correlation means an indicator orders windows the way the truth would, and says nothing about converting it into an estimate of the error. On the professor's data we will be able to choose, not to certify.

Finally, the coverage is limited in ways that should be stated: one dimension, sample sizes of five hundred and one thousand, targets that are all mixtures of standard families, and no target with both compact support and complex multimodality, which would be the worst case for the architecture.

11 Conclusions

Looking back at the three decisions that shaped the procedure, what stands out is that none was reached by improving the obvious answer. In each case it had to be given up.

The search for a better constant than $1.5\,\hat{\sigma}$ took more effort than any other part of this work, and it ended when we measured that no constant exists. That is a different kind of outcome from a negative result: it says the question was wrong, not that the answer was hard to find. The same shape recurs in the other two. Monotonicity came from removing the possibility of violation rather than from tuning a penalty until violations grew rare, and unit mass from a form whose mass is one before anything has been fitted.

The third decision had a consequence we did not anticipate. Requiring every reported quantity to be computable from the sample alone was adopted for a practical reason, since the method is meant for data whose distribution is unknown, and it turned out to be the most productive constraint in the project. It is what exposed the evaluation domain being taken from the true distribution, a dependency that had stayed invisible precisely because every figure was drawn on that same domain. And it is what forced the question of which sample-based indicators are informative, instead of reporting whichever ones happened to be available, which is how we found that the most natural of them points the wrong way.

What comes next is a question of dimension. The whole study is univariate, and the construction extends to several dimensions, since a convex combination of products of logistic distribution functions is again a valid joint distribution function; how many components that would need is open. The interactive tool that accompanies the project implements the design described here, with every choice above exposed as a control, so that taking the rejected road and watching it fail is something a reader can do rather than take on trust.

A Proofs

Throughout, $\sigma(t) = 1/(1+e^{-t})$ denotes the logistic function, whose derivative $\sigma'(t) = \sigma(t)(1-\sigma(t))$ is strictly positive. All parameters are finite.

Theorem 1 (saturation)

Let $F_\theta(x) = \sigma(u(x))$ with $u(x) = \sum_j w_j\,\sigma(a_j x + b_j) + c$.

Since σ takes values in $(0,1)$, each term $w_j\sigma(\cdot)$ lies between $\min(0, w_j)$ and $\max(0, w_j)$. Summing over the finitely many j and adding c,

$$m \;=\; c + \sum_j \min(0, w_j) \;\le\; u(x) \;\le\; c + \sum_j \max(0, w_j) \;=\; M,$$

with m and M finite. As σ is increasing, $\sigma(m) \le F_\theta(x) \le \sigma(M)$, and both bounds lie strictly inside $(0,1)$ because m and M are finite. Reaching 0 or 1 therefore requires $\|w\| \to \infty$. □

The limits themselves exist and are computable: as $x \to +\infty$ each $\sigma(a_j x + b_j)$ tends to 1 if $a_j > 0$ and to 0 if $a_j < 0$, so $F_\theta(+\infty) = \sigma(c + \sum_{a_j>0} w_j)$, and symmetrically at $-\infty$.

The hypothesis that matters is the boundedness of the hidden activation. With an unbounded one such as softplus the right tail becomes reachable, but the left does not, since softplus tends to 0 as its argument tends to $-\infty$ and the pre-activation still converges to c. The architecture discussed in Section 5.2 uses the logistic activation, so the bounded case is the relevant one.

Theorem 2 (validity by construction)

Let $F_\theta(x) = \sum_j \pi_j\,\sigma(a_j x + b_j)$ with $\pi_j > 0$, $\sum_j \pi_j = 1$ and $a_j > 0$.

Smoothness. A finite linear combination of compositions of C^∞ functions.

Monotonicity. $F'_\theta(x) = \sum_j \pi_j a_j \sigma'(a_j x + b_j)$ is a sum of strictly positive terms, so F_θ is strictly increasing.

Limits. As $x \to -\infty$ each $\sigma(a_j x + b_j) \to 0$ because $a_j > 0$, so $F_\theta \to 0$; as $x \to +\infty$ each tends to 1, so $F_\theta \to \sum_j \pi_j = 1$.

Mass. $\int_{\mathbb{R}} F'_\theta = F_\theta(+\infty) - F_\theta(-\infty) = 1$ by the fundamental theorem of calculus. □

The reparameterisation is what carries the theorem into practice. $\pi = \mathrm{softmax}(u)$ gives $\pi_j > 0$ with $\sum_j \pi_j = 1$ for every $u \in \mathbb{R}^J$, and $a = \mathrm{softplus}(\alpha)$ gives $a_j > 0$ for every $\alpha \in \mathbb{R}^J$. The hypotheses therefore hold on the whole parameter space, so no parameter value is inadmissible and no optimiser step can leave the set of valid distribution functions. In single precision there is one corner where this stops being literally true: $\mathrm{softplus}(\alpha)$ underflows to exactly zero below $\alpha \approx -104$, which turns a component into the constant $\frac{1}{2}$. Nothing in training drives a slope there, since the parameters start positive and the labels pull them further in that direction, but the guarantee is exact only in exact arithmetic.

One consequence should be stated because it is the limitation of Chapter 10: $F'_\theta > 0$ everywhere means the density is strictly positive on all of $\mathbb{R}$, so no distribution with compact support can be represented exactly.

Expressiveness

The family contains the estimator. Taking $J = n$, $\pi_j = 1/n$, $a_j = 1/h$ and $b_j = -x_j/h$ in (6) gives exactly the logistic Parzen estimator (3). Nothing is lost by the restriction at full capacity.

Density in the supremum norm. Let G be a continuous distribution function and $J \geq 1$. Put $\mu_j = G^{-1}((j - \frac{1}{2})/J)$, $\pi_j = 1/J$, and let all widths equal s. Write H for the step function with jumps $1/J$ at the μ_j. For $x \in [\mu_j, \mu_{j+1})$ we have $H(x) = j/J$ and $G(x) \in [(j - \frac{1}{2})/J, (j + \frac{1}{2})/J]$, so $\sup_x |H - G| \leq 1/(2J)$. As $s \to 0$ the mixture converges to H, the largest discrepancy being attained at the μ_j where the mixture takes the value $(j-1)/J + 1/(2J)$ against j/J, hence $1/(2J)$. By the triangle inequality

$$\limsup_{s \to 0} \sup_x |F_\theta(x) - G(x)| \;\leq\; 1/J,$$

so choosing $J \geq 1/\varepsilon$ suffices. □

Theorem 3 (equivariance)

Let S map a sample to an estimated distribution function. We want $S(\alpha x + \beta)(\alpha t + \beta) = S(x)(t)$ for $\alpha > 0$.

The hypothesis needed on the window is that it is itself equivariant, $h(\alpha x + \beta) = \alpha\, h(x)$. A window proportional to an equivariant scale satisfies this because $\hat\sigma(\alpha x + \beta) = \alpha\, \hat\sigma(x)$; so does the cross-validated selector actually used, since its criterion transforms by the factor $1/\alpha$ under the substitution and its minimiser therefore scales by α, which we confirm numerically to machine precision. With $h \mapsto \alpha h$ the kernel arguments satisfy

$$\frac{(\alpha t + \beta) - (\alpha x_i + \beta)}{\alpha h} = \frac{t - x_i}{h},$$

leaving the estimate and the leave-one-out labels unchanged.

For the network, if μ and s are equivariant, meaning $\mu(\alpha x + \beta) = \alpha\mu(x) + \beta$ and $s(\alpha x + \beta) = \alpha s(x)$, then the standardised sample $z = (x - \mu)/s$ is invariant. Every quantity computed from z is therefore identical: the initialisation, each gradient, each optimiser step and the fitted parameters. Evaluating the shifted model at $\alpha t + \beta$ standardises to $(t - \mu(x))/s(x)$, which is what the unshifted model receives at t. □

Without standardisation the property fails for two independent reasons: the initialisation does not depend on the data at all, so the model starts from the same function wherever the

sample lies, and the optimiser step is fixed in absolute parameter units while the parameters required scale with α.

The clamp adds mass

Let f be integrable on $[a, b]$ with $F(t) = F(a) + \int_a^t f$. Writing $f = f^+ - f^-$ with $f^+ = \max(f, 0)$ and $f^- = \max(-f, 0) \geq 0$,

$$\int_a^b f^+ \;=\; \int_a^b f \;+\; \int_a^b f^- \;=\; \big(F(b) - F(a)\big) + \int_a^b f^- \;\geq\; F(b) - F(a),$$

with equality if and only if $f \geq 0$ almost everywhere. Replacing a density by its positive part therefore breaks the relation between the two delivered curves unless the density was non-negative to begin with. □

Derivation of the two-stage plug-in

The asymptotically optimal window for density estimation is

$$h_{\mathrm{AMISE}} \;=\; \left[\frac{R(K)}{n\,\mu_2(K)^2\,\theta_4}\right]^{1/5}, \qquad \theta_4 = \int (f'')^2,$$

with $R(K) = 1/(2\sqrt{\pi})$ and $\mu_2 = 1$ for the Gaussian kernel. The unknown is θ_4, estimated by

$$\hat{\theta}_4(g) \;=\; \frac{1}{n^2 g^5} \sum_{i,j} \varphi^{(4)}\!\left(\frac{x_i - x_j}{g}\right), \qquad \varphi^{(4)}(u) = (u^4 - 6u^2 + 3)\,\varphi(u),$$

which needs its own pilot width g. The width minimising the asymptotic mean squared error of $\hat{\theta}_4$ is $g = [-2K^{(4)}(0)/(\mu_2\,\theta_6\,n)]^{1/7}$ with $K^{(4)}(0) = 3/\sqrt{2\pi}$; taking θ_6 from the normal reference, $\theta_6 = -15/(16\sqrt{\pi}s^7)$, gives

$$g \;=\; \left(\frac{96}{15\sqrt{2}}\right)^{1/7} s\,n^{-1/7} \;\approx\; 1.2407\,s\,n^{-1/7}.$$

As a check on the algebra, feeding the normal reference into the outer stage as well, with $\theta_4 = 3/(8\sqrt{\pi}s^5)$, gives $h = [4s^5/(3n)]^{1/5} = 1.059\,s\,n^{-1/5}$, which is Scott's rule. Numerically, on Gaussian samples the implemented selector returns an implicit constant of 1.043 at $n = 200$ and 1.051 at $n = 2000$, against the theoretical 1.06.

B Reproducibility

Every number quoted in this report comes from a script in the repository, and each script writes its output to a text file alongside it. The environment is Python 3.11 with NumPy, SciPy and PyTorch; the exact versions are pinned in `requirements.txt` and the test suite runs with `pytest`.

Where	Script
Ch. 2, generator checks and seeding	`sampling_quality.py`
Ch. 3 and 5, figures quoted in the text	`report_numbers.py`
Ch. 4, window selectors	`bandwidth_study.py`, `bandwidth_run.py`
Ch. 4, schedule exponent	`schedule_exponent.py`
Ch. 5, architecture comparison	`revalidate_torch.py`
Ch. 6, teacher window	`teacher_width.py`
Ch. 5–6, the MLP baseline	`crosscheck_real_code.py`
Ch. 8, domain and diagnostics	`pipeline_evidence.py`, `grid_rule.py`
Ch. 9, end-to-end comparison	`endtoend_compare.py`
Ch. 8 and 9, figures	`scripts/report_figures.py`
Ch. 10, other families	`resilience_non_gaussian.py`, `edges_capacity.py`

Table 14: Scripts live in `temp_analysis/` unless stated otherwise, and each writes its results to a text file of the same name. Two exceptions: the window study writes `bandwidth_n500.txt` and `bandwidth_n1000.txt`, and the figure script writes images into `results/`.

Two remarks on how the measurements are organised. Several of the studies in Chapters 4 and 8 were carried out with an independent NumPy implementation of the estimator rather than with the library, so that a defect in the code being assessed could not propagate into the assessment; the two implementations were checked to agree to machine precision on the estimator itself, and to within a few percent on the cross-validated window, where they use different candidate grids. Conversely, everything concerning the behaviour of the network was measured with the library itself, since that is what is being described.

The pipeline is run on a sample with the command given in Section 8.3, and the test suite with `pytest`. The tests include the properties asserted in Chapter 5: monotonicity and the exact limits under randomly perturbed parameters, agreement between the closed-form density and a numerical derivative of the distribution function, equivariance under translation and rescaling of the data, and the absence of any dependence of the estimation path on the module that generates test distributions.

C Decision log

A compact summary of the choices that define the procedure, with the evidence behind each.

Choice	Reason	Where
Window by least-squares cross-validation	Efficiency 1.26 mean, 1.59 worst, against 2.27 and 9.38 for $h_1 = 1.5\hat{\sigma}$	§4
Schedule $h_n = h_1/\sqrt{n}$ retained	Satisfies the consistency conditions; at a fixed budget it costs nothing	§4.6
Mixture of logistic CDFs	Valid distribution function at every parameter value; density in closed form	§5
Twelve components	Best worst case over the benchmark; the risk is asymmetric	Table 8
Standardised input	Without it a shift of 100 collapses the estimate to a constant	Thm. 3
No rectification, penalty or clamp	Nothing left to enforce; the clamp broke the density–CDF identity	§6.3
Domain $[\min x - 3\hat{\sigma},\ \max x + 3\hat{\sigma}]$	Worst-case mass outside 1.1×10^{-4} over sixteen densities	Table 10
Diagnostics: LSCV score, LOO likelihood, mass	Rank correlation 0.94 and 0.86 with the true error	Table 11
No comparison against the empirical CDF	Anticorrelated with the true error; rewards the narrowest window	Table 11

Table 15: The choices that define the procedure.